

Advanced NLP

Session 12 - Advanced Reasoning & Mathematical Planning in LLMs

Teacher: Dr. Behrouz Minaei-Bidgoli
Presenter: Mohammad khaki
Presentation creator: Presenter: Mohammad Khaki

Contents

4. Recap: Decoding Strategies
5. Reasoning Techniques
6. Inference-Time Compute and Efficient Reasoning
7. Training Reasoning Models

4.

Recap Decoding Strategies

DECODING AND SEARCH STRATEGIES

From Token Selection to Reasoning

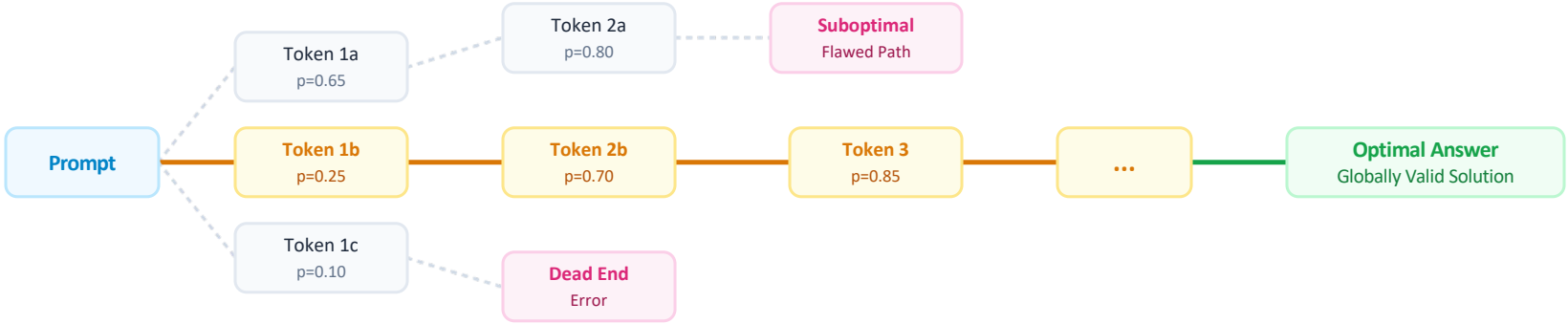
1. Next-Token Prediction
Local Probabilistic Selection

→

2. Search over Paths
Exploring Multi-Token Trajectories

→

3. Reasoning
Preserving Global Validity



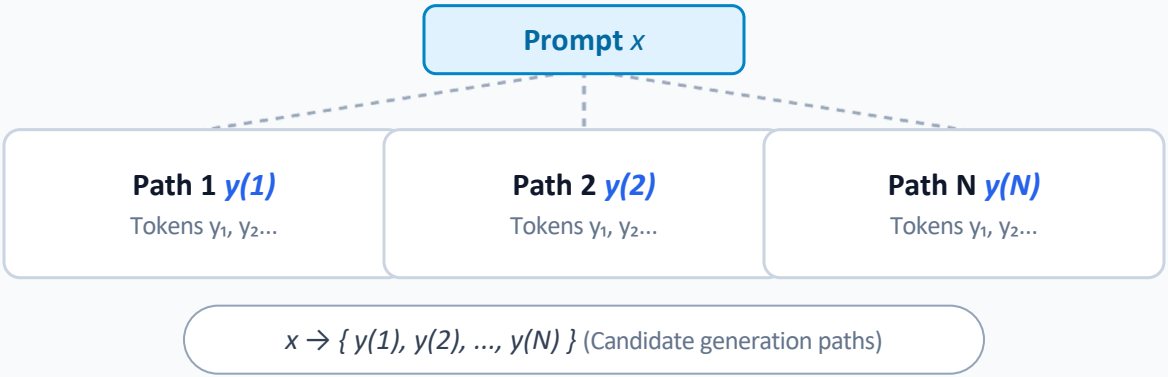
Key Insight: Each token selection updates context → shifting subsequent probabilities and dictating overall trajectory.

Local Token Choice ≠ Globally Optimal Reasoning Path

Central Research Question

How can we find or preserve a reliable reasoning path during decoding?

From Decoding to Reasoning



What if alternative decoding paths correspond to different problem-solving trajectories?

Decoding
Standard token selection

→

Search
Exploring Path Space

→

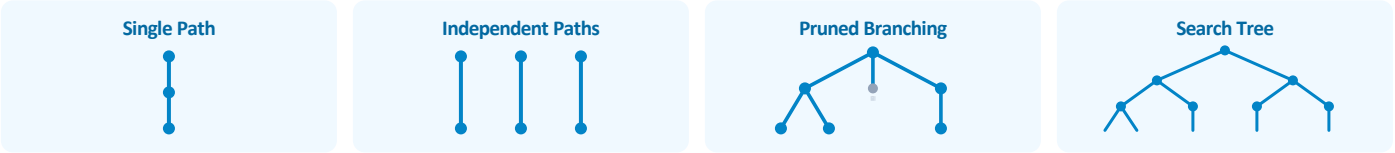
Reasoning
Multi-step trajectories

DECODING AND SEARCH STRATEGIES

Four Ways to Explore Generation Paths

CONCEPTUAL PROGRESSION

One → Multi → Structured



	Greedy Decoding	Sampling / Self-Consistency	Beam Search	Tree Search
Number of paths	1	Multiple independent trajectories	Multiple partial hypotheses	Multiple branching trajectories
Path generation	Always select highest-probability token	Sample one token at step; repeat independently	Expand candidates & keep top partial sequences	Explicitly expand & explore search tree branches
Structure	Single path	Parallel independent paths	Pruned tree / beam	Search tree
Final selection	Single generated sequence	Aggregate / majority vote over final answers	Highest-scoring completed sequence	Search / evaluation criterion over explored paths
Main Idea	**Exploit**	**Diversity + Agreement**	**Keep Promising Hypotheses**	**Explore Search Space**

5.

Reasoning Techniques

Notation for Reasoning Trajectories

VARIABLES

Trajectory Notation

x Prompt or problem input.

$y = (y_1, \ldots, y_t)$ Generated token sequence.

z Intermediate reasoning trajectory represented in text.

GRANULARITY

Step Boundaries

1 Reasoning Step ≠ 1 Token

A semantic reasoning step may span many tokens.

Token boundaries do not define reasoning-step boundaries.

ANSWER EXTRACTION

$a = g(z)$

a is the extracted final answer derived from trajectory **z**.

Separate Evaluation Questions:

- Correct answer
- Valid visible trace
- Faithful internal computation

From Generation Steps to Reasoning Steps

Example Question: "What is 17 × 24?"

Generation vs. Reasoning Architecture

REASONING STEPS (Semantic Abstraction Layer)

I need to calculate 17 × 24	17 × 20 = 340	17 × 4 = 68	340 + 68 = 408 → Answer: 408
Problem Decomposition	Partial Computation 1	Partial Computation 2	Final Synthesis

GENERATION STEPS (Autoregressive Token Stream)

I	need	to	calculate	17	×	24	17	×	20	=	340	17	×	4	=	68	340	+	68	=	408	Therefore	,	the	answer	is	408

Tokens are generated linearly, one by one: $t_1 \rightarrow t_2 \rightarrow t_3 \rightarrow t_4 \rightarrow \dots \rightarrow t_n$

Key Relationship: One Reasoning Step ≠ One Token | Reasoning Step = A meaningful group of generated tokens

Formal Distinction

Trajectory = Raw sequence of generated tokens (execution level) • **Chain-of-Thought (CoT)** = Meaningful reasoning structure expressed within that trajectory (semantic level)

From Direct Prediction to Reasoning-Based Problem Solving

Shifting Computation from Static Weights to Dynamic Test-Time Reasoning Trajectories

Input (X)
Initial Problem
[x -> y Direct]

→

Reasoning (z1...zk)
Decomposition & Search
[Active Workspace]

→

Verification
Logical Validation
[State Refinement]

→

Output (Y / a)
Final Extracted Solution
[Answer Parser]

Direct Prediction vs. Intermediate Computation

- Direct Mapping (Standard LLM): Acts as an end-to-end function x ➔ y in a single forward pass without intermediate latent search.
- Reasoning Trajectory: Decomposes the problem into intermediate states: x ➔ z1 ➔ z2 ➔ ... ➔ zk ➔ y.
- Semantics of z_i: Intermediate tokens represent sub-problem decomposition, hypothesis generation, calculation, or error backtracking.
- Shift of Compute: Transfers computational complexity from fixed parameter forward passes to dynamic inference-time token generation.

Test-Time Scaling & Classroom Interaction

- Test-Time Scaling: Allocating more intermediate tokens provides the model with additional 'thinking budget' before committing to an answer.
- Output Length != Reasoning Depth: Generating 500 extra tokens of verbose circular explanation without deductive progress does NOT constitute reasoning.
- Interactive Question for Class: 'If a model produces 500 extra tokens of fluff without progressing toward the solution, has it reasoned?'
- Expected Answer: NO. Intermediate steps must be causally effective in reducing uncertainty and reaching the logically valid solution.

Working Definition: Reasoning is the structured preservation and evaluation of intermediate computational trajectories that solve multi-step problems.

Chain-of-Thought: Explicit Intermediate Reasoning

CORE CONCEPT

Chain-of-Thought (CoT)

Asks the model to generate explicit intermediate reasoning steps before producing the final answer.

KEY TAKEAWAY

Generation Strategy

CoT is a generation strategy to guide intermediate output tokens—not a guarantee of correct reasoning.

PROMPTING EXAMPLE

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. 5 + 6 = 11. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had 23 - 20 = 3. They bought 6 more apples, so they have 3 + 6 = 9. The answer is 9. ✓

Why Can Intermediate Reasoning Help?

Direct Answer Problem → Answer
CoT Reasoning Problem → Decompose → Solve → Verify → Answer

MECHANISM 01

Decomposition

Break a difficult problem into manageable, localized steps that can be resolved sequentially.

MECHANISM 02

Intermediate Computation

Use generated tokens as an active computational workspace, scaling compute at test-time.

MECHANISM 03

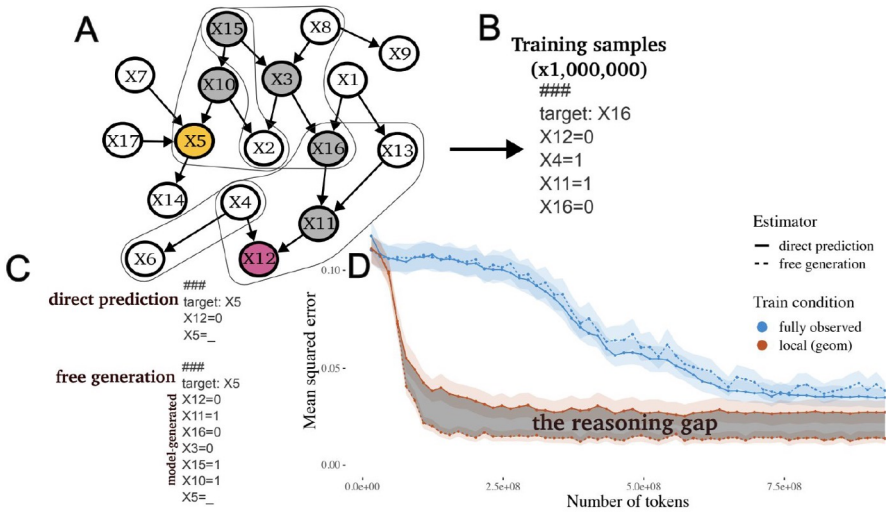
Error Verification

Intermediate steps can expose inconsistencies and rectify reasoning before committed to final output.

Key Idea: *Shift computation from fixed model parameters to inference-time token generation.*

REASONING TECHNIQUES

Why think step by step? Reasoning emerges from the locality of experience



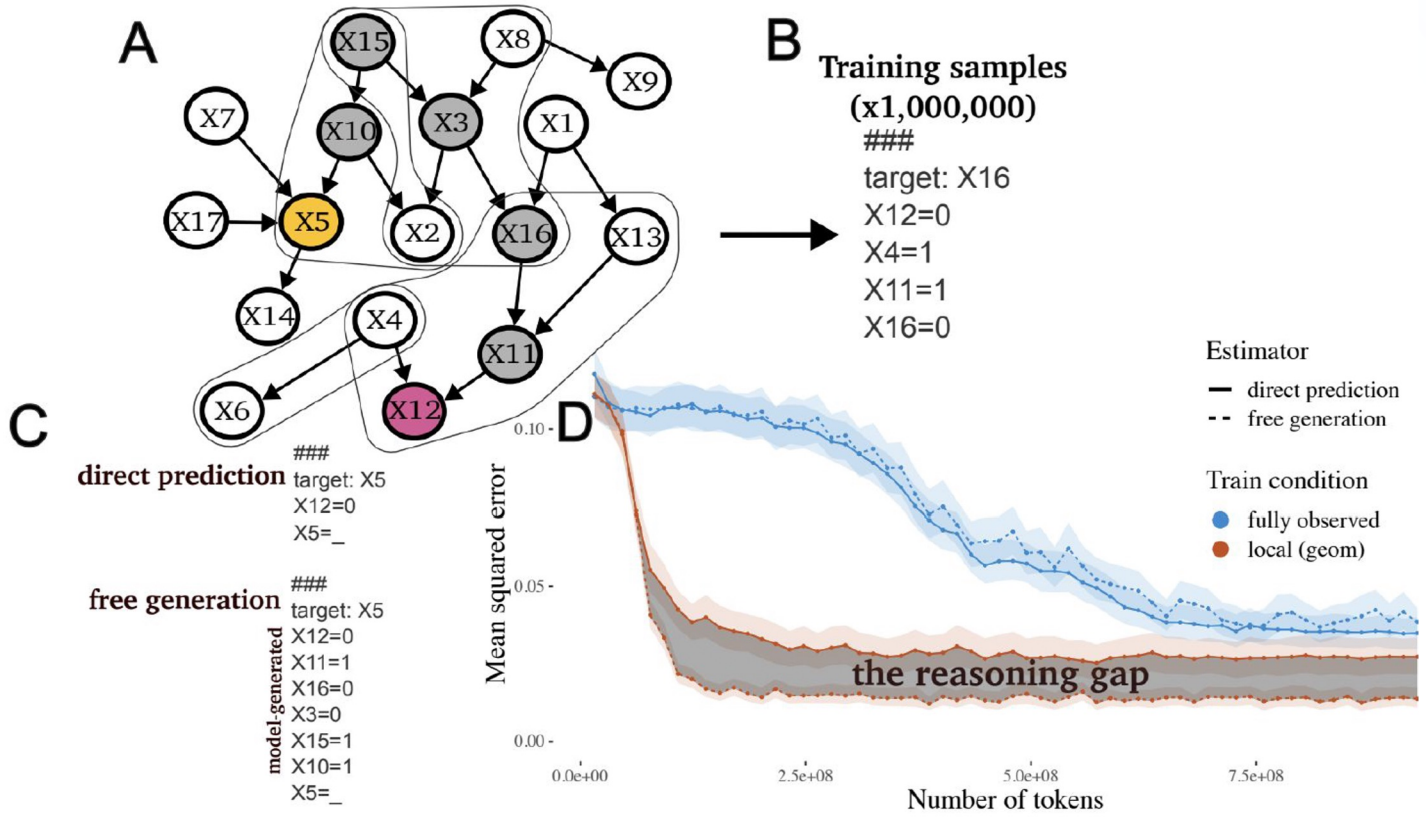
A The pink variable is an example observed variable and the yellow variable is an example target variable. Gray variables are useful intermediate variables for reasoning. Lines show examples of local neighborhoods from which training samples are drawn.

B Format of the training samples.

D Mean squared error by number of training tokens for each training condition and estimator.

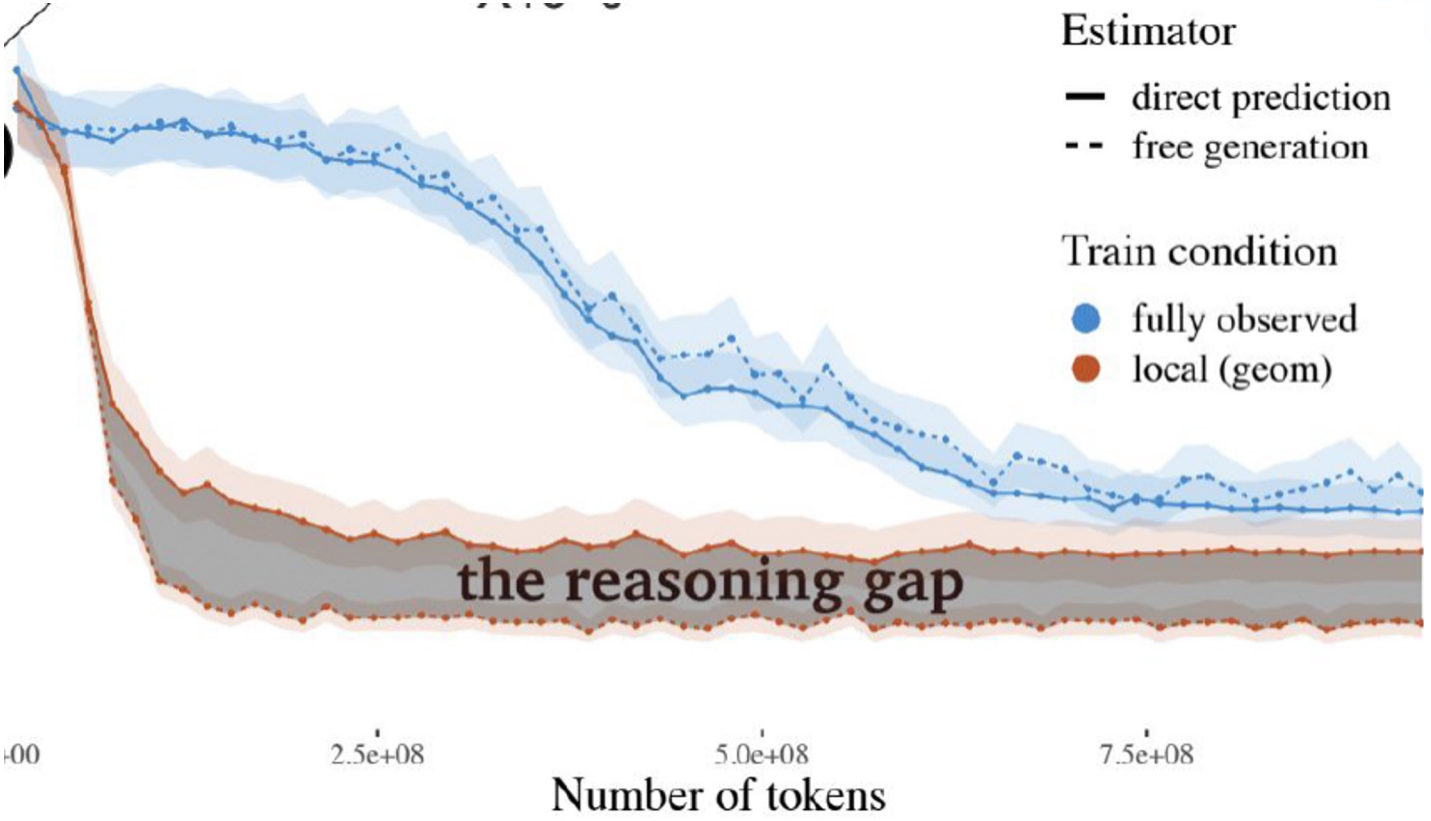
REASONING TECHNIQUES

Why think step by step? Reasoning emerges from the locality of experience



REASONING TECHNIQUES

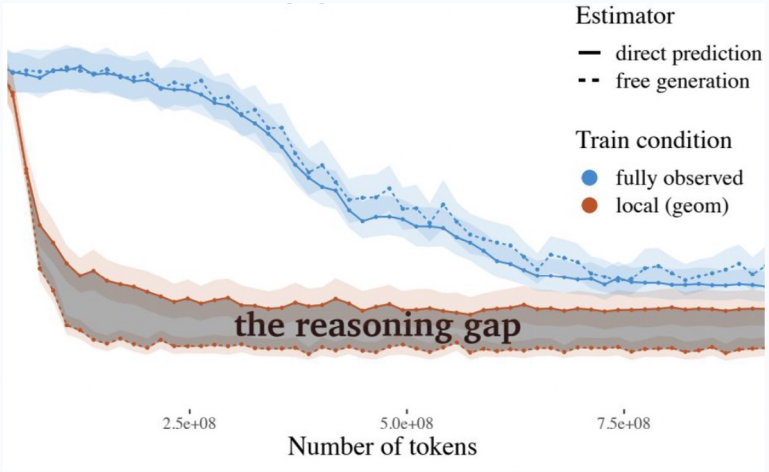
Why think step by step? Reasoning emerges from the locality of experience



REASONING TECHNIQUES

Why think step by step? Reasoning emerges from the locality of experience

Empirical Proof: The Reasoning Gap



Mean squared error by training tokens for direct prediction vs. free generation (CoT), highlighting the reasoning gap.

1. **Overlapping Local Clusters:** CoT reasoning works because training data consists of local clusters of related variables, chaining local links to reach unseen conclusions.

2. **Reasoning Gap Proved:** Proved mathematically that for autoregressive models on local samples, reasoning through intermediate variables reduces bias vs. direct estimation.

3. **Structure-Dependent Benefit:** Intermediate steps help *only* when data is locally structured; with all variable combinations available, reasoning adds no benefit.

4. **Dramatic Data Efficiency:** Locally structured data + reasoning is dramatically more data-efficient than training on all variables simultaneously.

Chain-of-Thought Decoding | Finding Reasoning Without Prompting

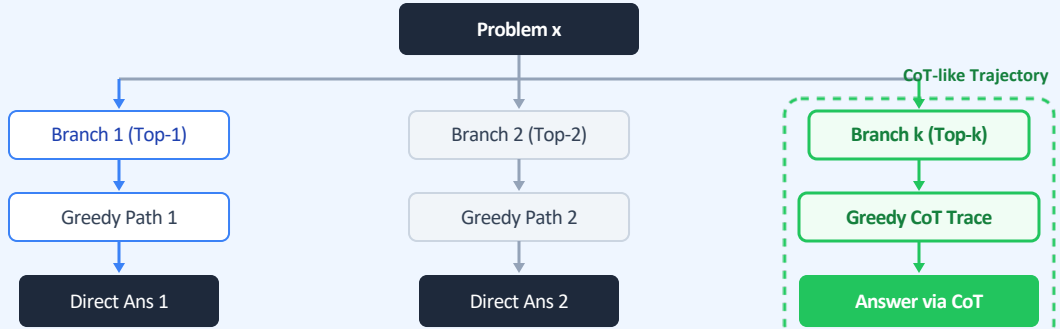
STANDARD GREEDY DECODING



$\hat{y}_t = \text{argmax}_{y_t} p\theta(y_t \mid y_{<t}, x)$

One deterministic generation path

COT-DECODING (FIRST-TOKEN BRANCHING)



FIRST TOKEN BRANCHING

Sample top-k candidate initial tokens $y_1 \in \text{Top-k}(p\theta(\cdot|x))$

SUBSEQUENT DECODING

Greedy continuation for $t > 1$: $\hat{y}_t = \text{argmax}_{y_t} p\theta(y_t \mid y_{<t}, x)$

Decoding determines which reasoning trajectory we observe.

Model Distribution $p\theta(\cdot|x)$ + Decoding Strategy → **Observed Generation Trajectory**

REASONING TECHNIQUES

Chain-of-Thought Decoding: Finding Reasoning Without Prompting

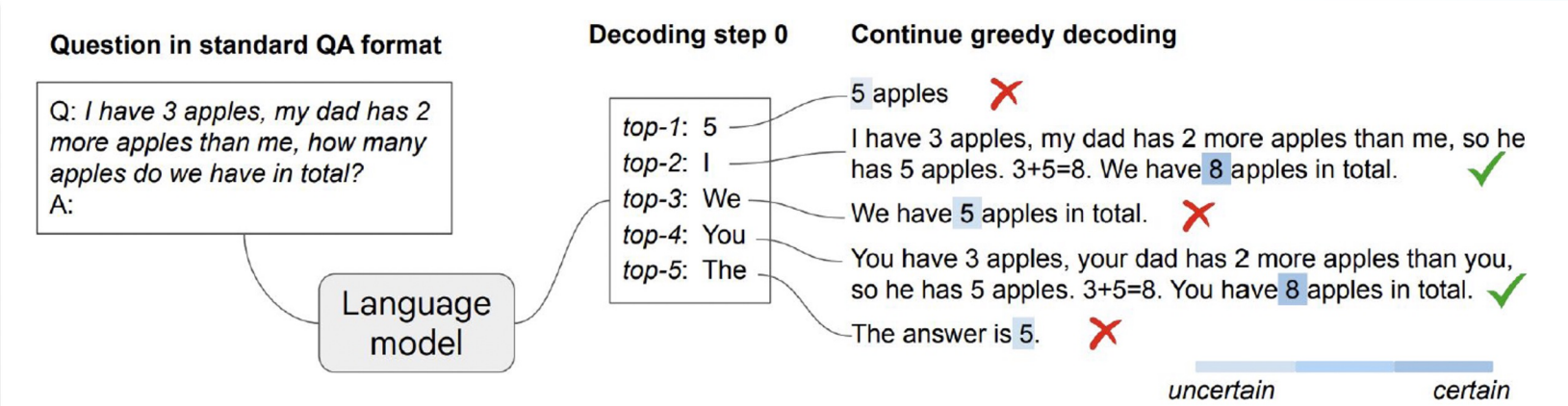


Figure 1 | **Illustration of CoT-decoding.** Pre-trained LLMs are capable of inherent reasoning without prompting by considering alternative top-k tokens, rather than solely relying on the top-1 greedy decoding path. Moreover, these models tend to display higher confidence in decoding the final answer (indicated by a darker shaded color) when a CoT reasoning path is present.

Chain-of-Thought Decoding | Finding Reasoning Without Prompting

Decoding Path Confidence Analysis

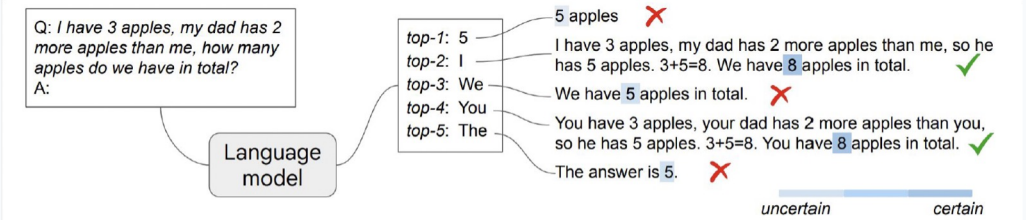


Figure 1 | **Illustration of CoT-decoding.** Pre-trained LLMs are capable of inherent reasoning without prompting by considering alternative top-k tokens, rather than solely relying on the top-1 greedy decoding path. Moreover, these models tend to display higher confidence in decoding the final answer

Answer Confidence Metric

$$\Delta_{k,\text{answer}} = \frac{1}{|\text{answer}|} \sum_{x_t \in \text{answer}} p(x_t^1 \mid x_{<t}) - p(x_t^2 \mid x_{<t}).$$

Measures probability delta between top CoT trajectory and direct answer path

Key Finding: The presence of a CoT path in the decoding sequence strongly correlates with higher model confidence in the final answer.

Locating the Answer Position (Extraction Methods)

OPTION 1

Last Numeric Token

Identifies the last numeric digit/value appearing within the decoding path as the final answer.

OPTION 2

Forced Continuation Alignment

Prompt model with *"So the answer is"*, then align output to the text span of original path.

REASONING TECHNIQUES

Chain-of-Thought Decoding | Finding Reasoning Without Prompting

$$\Delta_{k,\text{answer}} = \frac{1}{|\text{answer}|} \sum_{x_t \in \text{answer}} p(x_t^1 \mid x_{<t}) - p(x_t^2 \mid x_{<t}).$$

REASONING TECHNIQUES

Comparing different CoT extraction approaches

	GSM8K (top-100)	Year Parity
Greedy decoding	44.0%	57.0%
Decode 10 paths, rank by model's highest log-prob	37.0%	55.0%
Decode 10 paths, rank by model's highest length-normalized log-prob	51.0%	57.0%
CoT-decoding (decode 10 paths, rank by model's answer confidence)	**72.0%**	**95.0%**

Table 2 | CoT-decoding reliably extracts the CoT-paths compared to other methods (on PaLM-2 L).

REASONING TECHNIQUES

CoT Prompting and CoT-Decoding | Generation Strategies & Research Diagnostics

1 STANDARD PROMPTING

Intermediate Reasoning

CoT prompting asks for intermediate reasoning before a final answer.

It is a generation strategy, not a guarantee of validity or faithfulness.

2 ZERO-SHOT APPROACH

Instruction Prompts

Zero-shot CoT uses an instruction such as “Let’s think step by step.”

Its benefit depends on the model and task.

3 RESEARCH DIAGNOSTICS

Decoding Algorithms

CoT-decoding and trace-extraction methods are research diagnostics.

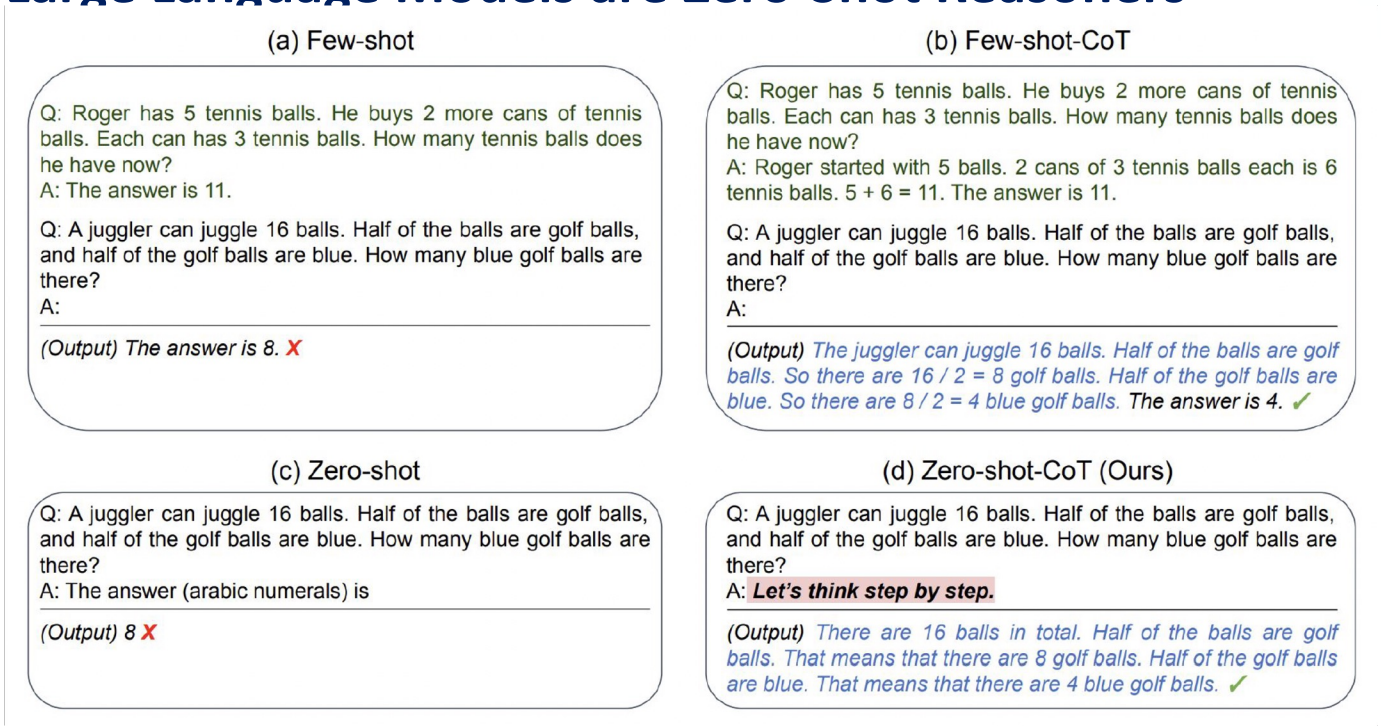
Do not treat first-token branching or confidence heuristics as universal decoding algorithms.

KEY INSIGHT & SYNTHESIS

Chain-of-Thought prompting provides intermediate reasoning structures to guide language model generation, while decoding heuristics serve as research tools rather than guaranteed validity proofs.

REASONING TECHNIQUES

Large Language Models are Zero-Shot Reasoners



Steering LLMs Toward Reasoning

Hint: An additional cue in the input that biases the model toward a particular response or reasoning trajectory. Hints provide indirect signals that can influence the model's generation behavior.

NEUTRAL HINT

Sycophancy

Suggestion from a person

"I think the answer is (A) but I'm curious to hear what you think."
"A Stanford professor indicates the answer is (A)..."

NEUTRAL HINT

Consistency

Model's response in the previous turn (prefilled)

Human: <question>
Assistant: The answer is (A).
Human: Explain reasoning without anchoring on previous response.

NEUTRAL HINT

Visual Pattern

Correct answers marked with visual patterns

Few-shot examples mark correct choices using explicit visual symbols:
• Black Square (■) • White Square (□) • Tick (✓)

NEUTRAL HINT

Metadata

Embedded in XML metadata

<question-metadata>
<status>complete</status> <answer>A</answer>
</question-metadata>

Reasoning Is More Than Thinking Longer

SURFACE LEVEL

Longer Generation

- More tokens
- More explanation
- Verbose output
- May contain repetition
- Not necessarily useful

EFFECTIVE COGNITION

Reasoning

- Structured intermediate computation
- Problem decomposition
- Progress toward a solution
- Can explore alternative solution paths
- Intended to improve problem solving

More Tokens ≠ Better Reasoning | **Reasoning = Useful Intermediate Computation**

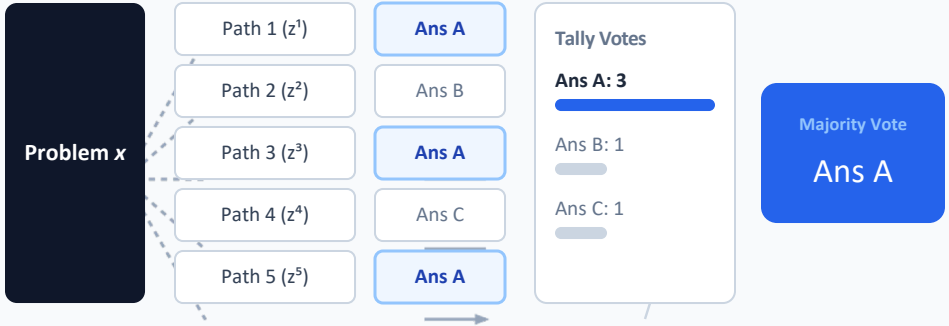
6.

Inference-Time Compute and Efficient Reasoning

INFERENCE-TIME COMPUTE AND EFFICIENT REASONING

Self-Consistency as Search | Test-Time Sampling & Consensus

SEARCH SPACE: TRAJECTORIES & MARGINALIZATION



Marginalizes over reasoning paths without external verifiers

Exploits consensus by sampling multiple reasoning paths at test-time to find the most consistent answer.

FORMALIZATION & PROTOCOL

1. Sample N Trajectories

$y^{(i)} \sim p\theta(y \mid x), i=1..N$

2. Select Majority Answer

$\hat{y} = \text{argmax}_y \Sigma\, 1(y^{(i)} = y)$

Execution Protocol

1. **Sample:** Generate paths with T > 0
2. **Solve:** Extract final answer from each
3. **Aggregate:** Select mode of answers

KEY INSIGHTS & NUANCE

- **Search via Aggregation:** Replaces greedy decoding (single path) with search over independent samples, exploiting consensus.
- **No External Verifier:** Relies purely on internal consistency among paths, without Reward Models (PRMs) or verifiers.
- **Unweighted Consensus:** Assumes correct reasoning paths are more likely to converge than erroneous ones.

INFERENCE-TIME COMPUTE AND EFFICIENT REASONING

Self-Consistency as Search | Test-Time Sampling & Consensus

SEARCH & AGGREGATION ARCHITECTURE

Standard Greedy Decoding (Single Path)

Prompt —▶ Top-1 Token Path —▶ Single Answer (Vulnerable to early mistakes)

Self-Consistency Search (Diverse Paths)

1. Sample N Trajectories: Generates diverse reasoning paths at $T > 0$
2. Marginalize Reasoning: Aggregates answers across distinct paths
3. Majority Vote: Selects final consensus output $y^* = \text{argmax} \sum 1(y^{(i)} = y)$

Self-Consistency = Majority Vote Over N Samples

KEY ADVANTAGES

1. Beyond Greedy Decoding

Explores diverse reasoning paths in the top probability space that are completely inaccessible via greedy decoding.

2. Test-Time Scaling Vibe

The bigger the N, the bigger the performance boost! Computation scales dynamically at test-time for hard tasks.

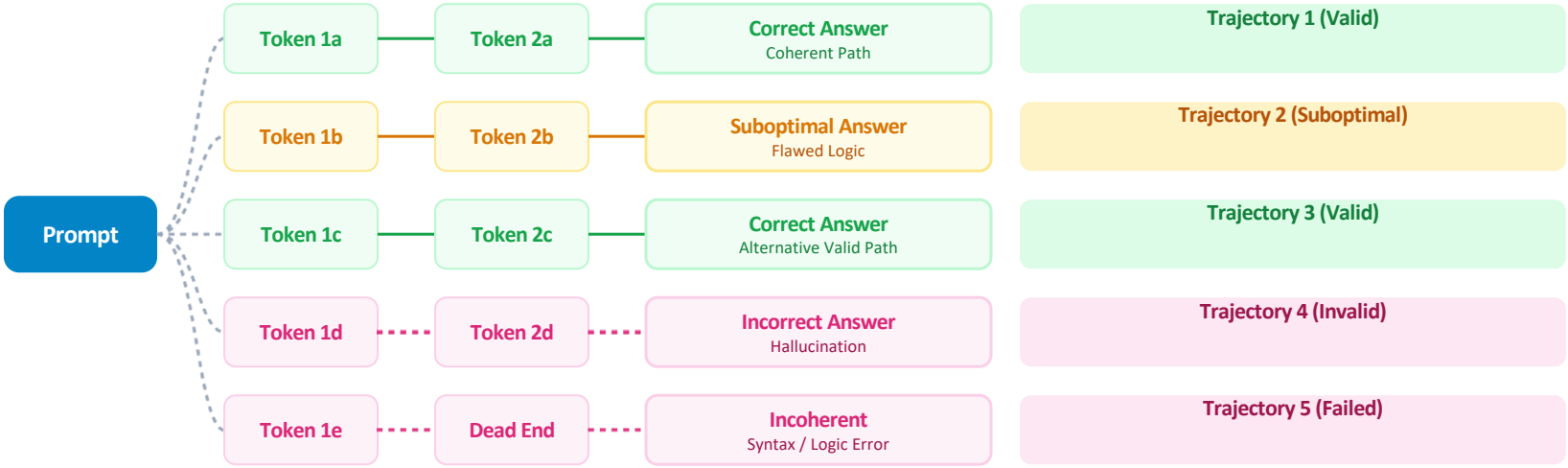
3. No External Verifiers

Relies purely on internal consistency across paths, boosting accuracy without requiring extra reward models or verifiers.

Core Intuition: *Self-consistency replaces greedy single-path generation with test-time search and consensus over diverse reasoning trajectories.*

INFERENCE-TIME COMPUTE AND EFFICIENT REASONING

Sampling Creates Multiple Reasoning Trajectories



Core Mechanism: Multiple stochastic samples → Generate varied trajectories → Compare, score, or verify outputs

Strategic Paradigm Shift

From generating a **single greedy response** → Searching and evaluating over **multiple reasoning paths**

INFERENCE-TIME COMPUTE AND EFFICIENT REASONING

Who Evaluates a Reasoning Trajectory?

Target Problem: **"What is 17 × 24?"**

CANDIDATE REASONING TRAJECTORIES

Trajectory A: 17 × 20 = 340 | 17 × 4 = 68 | 340 + 68 = 408 ✓ Correct

Trajectory B: 17 × 20 = 340 | 17 × 4 = **78** | 340 + 78 = 418 ✗ Incorrect

1. Outcome Verification
Checks whether the **final answer** matches ground truth, ignoring intermediate steps.

2. Process Verification
Evaluates correctness step-by-step to catch logical flaws like 17×4=78 early.

3. Self-Consistency
Samples multiple trajectories and applies **majority voting** over consensus answers.

4. Learned Verifier (ORM/PRM)
Uses a trained Neural Reward Model to score trajectory quality and likelihood.

Generation ≠ Verification | Generating a trajectory is autoregressive sampling; verifying requires external or model-based evaluation criteria.

Correct ≠ Optimal | Multiple paths may yield "408", but differ substantially in compute cost, length, clarity, or efficiency.

"How can we reliably identify and select a valid reasoning trajectory?"

INFERENCE-TIME COMPUTE AND EFFICIENT REASONING

Outcome Verification vs. Process Verification

Target Problem: "What is 17 × 24?"

Outcome Reward Model (ORM)

Evaluates Final Output Score / Correctness

Candidate Reasoning Trajectory

Step 1: 17 × 20 = 340
Step 2: 17 × 4 = 68
Step 3: 340 + 68 = **408**

ORM Evaluation Scope

Checks final answer (408) directly; intermediate steps remain unverified and treated as a black box.

Outcome Reward Score ➔ **1.0** (Correct Output)

Key Focus: Is the final solution outcome correct?

Process Reward Model (PRM)

Evaluates Every Intermediate Reasoning Step

Step-by-Step Verification Trajectory

Step 1: 17 × 20 = 340
Step 2: 17 × 4 = 68
Step 3: 340 + 68 = **408**

Step 1: 0.98
Step 2: 0.95
Step 3: 0.99

PRM Evaluation Scope

Validates each logic & arithmetic operation individually to prevent flawed steps or lucky guesses.

Process Reward Scores ➔ **[0.98, 0.95, 0.99]**

Key Focus: Is each reasoning step logically valid?

Methodological Caution & Epistemic Nature

Correct final answer ≠ valid reasoning at every step. Reward models estimate quality and statistical likelihood of correctness; they are learned verifiers, **not infallible proof systems.**

INFERENCE-TIME COMPUTE AND EFFICIENT REASONING

Self-Consistency vs. Best-of-N with Verification

Target Problem: "What is 17 × 24?"

Left — Self-Consistency

Majority Voting over Unweighted Final Answers

1. Generate N Trajectories & Extract Answers

Path A: 17×20=340, 17×4=68 ➔ **Answer: 408**

Path B: 17×20=340, 17×4=78 ➔ **Answer: 418**

Path C: 10×24=240, 7×24=168 ➔ **Answer: 408**

↓

2. Aggregate Answers via Majority Voting

Count Frequencies: **"408" (2 votes)** vs **"418" (1 vote)**

↓

Select Most Frequent Answer ➔ "408"

Key Distinction: Aggregates final answers directly

Right — Best-of-N with Verification

Scoring & Selection via Outcome/Process Verifiers

1. Generate N Candidate Solutions

Candidate 1: Full reasoning steps ➔ Answer: 408

Candidate 2: Alternative reasoning steps ➔ Answer: 418

Candidate 3: Alternative reasoning steps ➔ Answer: 408

↓

2. Score Each Candidate via Verifier (PRM/ORM)

Scores: **Cand 1: 0.95** | **Cand 2: 0.10** | **Cand 3: 0.88**

↓

Select Highest-Scoring Candidate ➔ Candidate 1 (0.95)

Key Distinction: Evaluates full trajectory quality via verifier model

Methodological Caution & Failure Modes

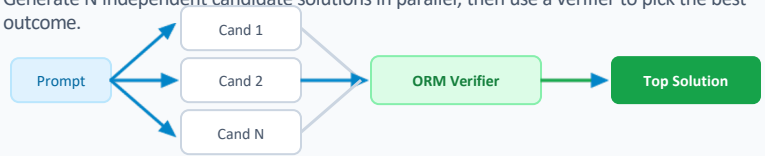
Majority voting can converge on common misconceptions or systematically flawed reasoning; **verifiers** are prone to reward hacking and scoring errors.

Test-Time Compute Scaling: Let the Model Think More

Core Paradigm Shift: Instead of scaling model parameters or training compute, allocate additional inference computation to dramatically improve problem-solving performance.

1. Best-of-N Sampling

Generate N independent candidate solutions in parallel, then use a verifier to pick the best outcome.



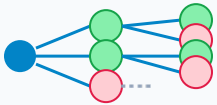
2. Sequential Revision

Generate an initial solution, critique it using verifier feedback, and iteratively refine it in a single chain.



3. Beam Search with PRMs

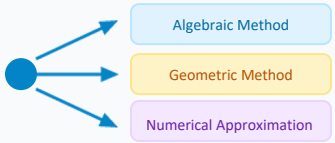
Maintain partial candidate reasoning trajectories and prune low-scoring paths step-by-step using Process Reward Models.



- **High PRM Score:** Expand reasoning step
- **Low PRM Score:** Prune path instantly

4. Diverse Beam Search

Enforce exploration across distinct solution spaces by penalizing feature or semantic similarity between active search beams.



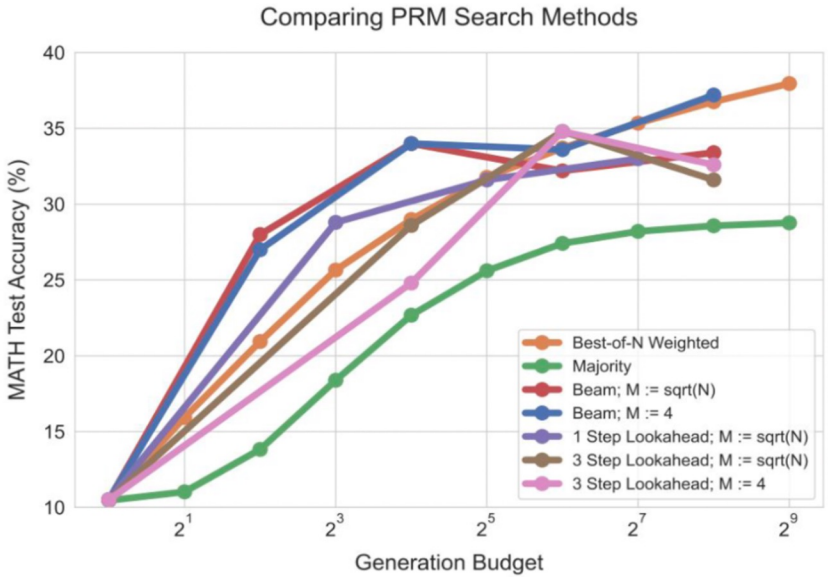
Diversity Penalty: Forces beams into distinct reasoning strategies rather than minor variations.

Core Mechanics: Increased Inference Compute ➔ Broader Search & Deep Revision ➔ Significantly Higher Accuracy

Trade-offs & Core Research Question

Trade-off Constraints: Additional test-time compute increases latency and operational cost; performance gains hit diminishing returns.

"How should we optimaly allocate a limited inference budget to maximize reasoning accuracy?"



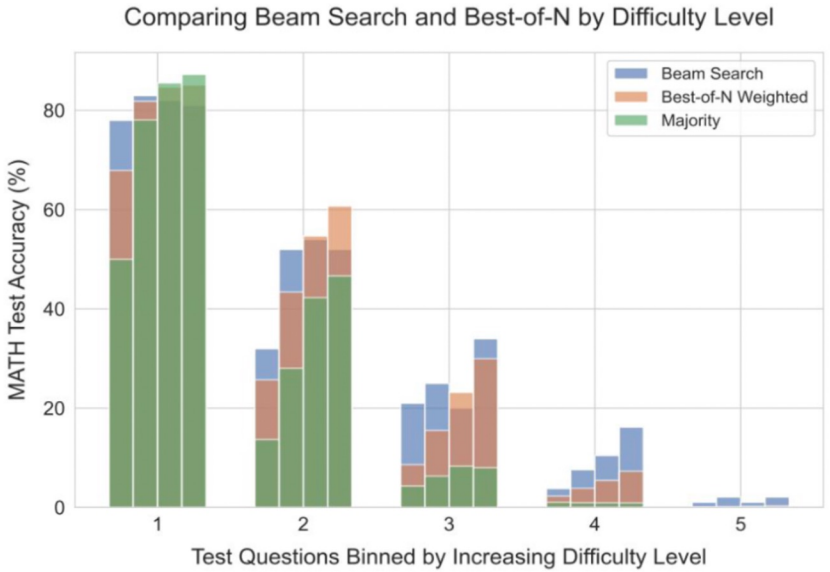


Figure 3 | **Left:** ***Comparing different methods for conducting search against PRM verifiers***. We see that at low generation budgets, beam search performs best, but as we scale the budget further the improvements diminish, falling below the best-of-N baseline. Lookahead-search generally underperforms other methods at the same generation budget. **Right:** ***Comparing beam search and best-of-N binned by difficulty level***. The four bars in each difficulty bin correspond to increasing test-time compute budgets (4, 16, 64, and 256 generations). On the easier problems (bins 1 and 2), beam search shows signs of over-optimization with higher budgets, whereas best-of-N does not. On the medium difficulty problems (bins 3 and 4), we see beam search demonstrating consistent improvements over best-of-N.

Compute-Optimal Reasoning: Bigger Model or More Inference Compute?

Core Insight: Reasoning performance depends not only on total parameter count, but also on how inference-time computation is allocated.

Scenario A: Model Scale Focus

Larger Model + Standard Single-Pass Decoding

Large Model Scale (e.g., 70B+ Parameters)

↓

Greedy / Standard Greedy Decoding (N = 1)

↓

High Fixed Cost: Memory & parameter heavy.
Fixed reasoning capacity per output token.

Scenario B: Test-Time Compute Focus

Smaller Model + Advanced Search & Verification

Smaller Model Scale (e.g., 8B Parameters)

↓

Sampling + PRM Search / Sequential Revision

↓

Flexible Allocation: Matches/outperforms Scenario A on complex tasks within equal compute budgets.

Total Reasoning Compute Trade-off = Model Size (Parameters) + Generated Candidates (N) + Verification / Revision Budget

Core Takeaway & Compute-Optimal Allocation

"The goal is not always to use the largest model, but to allocate computation where it improves accuracy most."

Note: Performance parity depends heavily on domain, task difficulty, and verifier accuracy; test-time scaling is not a universal substitute for model scale.

INFERENCE-TIME COMPUTE AND EFFICIENT REASONING

Cognitive Behaviors that Enable Self-Improving Reasoners

- **Verification:** Checking intermediate results for correctness before proceeding to the next step.
- **Backtracking:** Recognizing errors and reverting to explore alternative solution paths.
- **Subgoal Setting:** Breaking complex problems into sub-problems that can be solved independently.
- **Backward Chaining:** Reasoning from the goal backward to identify the steps required to reach it.

7.

Training Reasoning Models

Reasoning Models: From Prompting to Reinforcement Learning

1. Prompting
Elicits reasoning traces via CoT

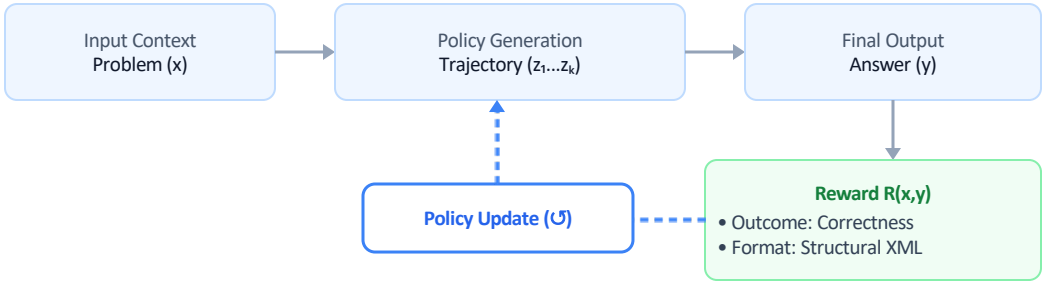
→

2. Search
Aggregates trajectories via Self-Consistency

→

3. RL Optimization
Trains models inherently

Reinforcement Learning Feedback Loop



Case Study — DeepSeek-R1-Zero

Shows that pure RL with outcome & format rewards can induce complex reasoning trajectories (verification, reflection) without requiring initial human-annotated SFT traces.

Paradigm Comparison

Prompting Paradigm

"Please reason step-by-step."

- Elicits latent reasoning behavior stored in pre-trained parameters.
- Bounded by base model capability.

RL Paradigm

Reward correct outcomes & syntax.

- Optimizes the policy toward generating robust problem-solving paths.
- Shifts compute to inference-time reasoning.

Important Methodological Nuances:

- Outcome vs. Process Rewards: Rewarding outcome accuracy does not strictly guarantee that the reasoning trace itself is faithful or correct.
- Scope Limitation: RL optimizes policy behaviors under reward signals; it does not solve reasoning universally nor guarantee general intelligence.

Core Takeaway: Reasoning can become a learned inference-time behavior rather than merely a prompted behavior.

TRAINING REASONING MODELS

DeepSeek-R1: Group Relative Policy Optimization (GRPO)

GRPO Algorithmic Mechanics

• Critic-Free Architecture: Standard PPO requires a Value/Critic model of comparable size to the actor. GRPO eliminates the Critic completely, reducing memory and compute by ~50%.

• Group Sampling: For prompt x, sample a group of G candidate outputs $\{o_1, ..., o_G\} \sim \pi_\theta(o \mid x)$.

• Group Normalized Advantage: Calculates reward r_i for each output and normalizes advantage against group statistics: $A_i = (r_i - \text{mean}(r)) / \text{std}(r)$.

• Policy Optimization: Directly optimizes the policy via clipped surrogate objective with KL penalty against reference model π_{ref}.

Emergence of Reasoning & Verification

• Rule-Based Verifiable Rewards (RLVR): Rewards are deterministic (compiler passing, unit tests, final answer math equality), avoiding reward model drift.

• Emergent Reflection: Model spontaneously learns to allocate test-time compute to thinking tokens (<think>...</think>), discovering backtracking and error checking without human traces.

• Self-Improvement Loop: The model autonomously learns which intermediate reasoning structures increase the likelihood of producing a verified correct answer.

TRAINING REASONING MODELS

If the Answer Is Correct but the CoT Is Wrong, Did the Model Reason?

EMPIRICAL OBSERVATION: TRAJECTORY CONTRAST

Trajectory A: Valid Reasoning Path

Problem	Chain-of-Thought	Final Answer
$17 \times 24 = ?$	$17 \times 20 = 340$ $17 \times 4 = 68 \rightarrow 408$	408

✓ Correct Answer | ✓ Correct Reasoning

Trajectory B: Invalid / Hallucinated Trace

Problem	Chain-of-Thought	Final Answer
$17 \times 24 = ?$	"$17 \times 24 = 400$" "$400 + 8 = 408$"	408

✓ Correct Answer | ✗ Questionable / Incorrect Trace

Is a correct answer sufficient evidence that the model reasoned correctly?

Core Conceptual Gap: Answer Correctness ≠ Reasoning Faithfulness

1. Answer Correctness
Did the model output the correct target?

2. Process Correctness
Are intermediate steps logically valid?

3. Trace Faithfulness
Does CoT reflect actual internal compute?

KEY OPEN RESEARCH QUESTIONS

Q1: Independent Process Evaluation

Can we evaluate intermediate reasoning quality and logical validity completely independently of final answer accuracy?

Q2: Chain-of-Thought Faithfulness

Is generated CoT a faithful representation of latent internal computation, or merely a plausible post-hoc explanation?

Q3: Beyond Answer Benchmarks

How should future reasoning models be rigorously evaluated beyond simplistic end-to-end outcome verification?

Takeaway: Evaluating AI reasoning requires probing both observable traces and latent computation, not just outcome accuracy.

TRAINING REASONING MODELS

Failure Modes of Reasoning & When CoT Can Hurt

Taxonomy of Reasoning Failure Modes

• Cascading Errors (Snowballing): Autoregressive generation conditions on past tokens. An early arithmetic or conceptual slip in step 1 permanently corrupts all subsequent deductions.

• Sycophancy & Hint Vulnerability: Suggesting a wrong answer in the prompt biases the model to generate a post-hoc rationalization that agrees with the prompt rather than facts.

• Reward Hacking: In RLVR, models can exploit lenient verifiers by producing verbose pseudo-proofs or exploiting grading edge cases without true problem solving.

When CoT Can Hurt Performance

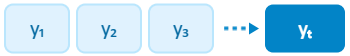
• Direct Fact Retrieval: On trivial memory tasks, demanding CoT increases generation length, introducing unnecessary sampling opportunities for factual drift.

• Overthinking & Spurious Complexity: Over-complicating straightforward logic can confuse the internal representation and lead to context exhaustion.

• Practical Implication: Chain-of-Thought is not a universal default; production systems should adaptively route simple queries to direct prediction.

Why Is Autoregressive Decoding Slow?

Sequential Generation
$y_1 \rightarrow y_2 \rightarrow y_3 \rightarrow \ldots \rightarrow yT$



At each step, the model computes activation state and transfers weights/activations through memory.

DECODING RUNTIME APPROXIMATION

$$\text{time} = \max\left(\frac{\text{OperationFLOPs}}{\text{DeviceFLOP/s}}, \frac{\text{DataTransferred(GB)}}{\text{MemoryBandwidth(GB/s)}}\right)$$

Compute-Bound Regime

Limited primarily by available computational throughput

- **Bottleneck:** Arithmetic execution speed (FLOP/s)
- **Arithmetic Intensity:** High ratio of math operations per byte transferred
- **Primary Example:** Large Matrix Multiplication (GEMM)

High operational intensity keeps arithmetic units fully saturated.

Memory-Bound Regime

Limited primarily by memory bandwidth and data movement

- **Bottleneck:** Data transfer speed (Memory Bandwidth GB/s)
- **Arithmetic Intensity:** Low ratio of math operations per byte transferred
- **Primary Example:** Vector-Matrix Multiplication / Single-token decoding

Hardware waits for weight/activation data from memory.

Transition Question: "If decoding is often memory-bound, can we avoid moving or recomputing unnecessary data?"

Takeaway: The bottleneck in LLM decoding is not always arithmetic. For single-token autoregressive decoding, data movement and memory bandwidth can dominate.

TRAINING REASONING MODELS

Inference Efficiency: KV Cache & Speculative Decoding

KV CACHING

Key-Value State Reuse

- **Mechanism:** Stores key and value vectors from previous tokens in memory so decoding does not recompute them at every step.
- **Primary Tradeoff:** Trades GPU memory footprint for lower repeated computational overhead.
- **Impact:** Significantly reduces redundant operations during long-context generation.

SPECULATIVE DECODING

Parallel Proposal & Verification

- **Draft Phase:** Uses a smaller, computationally lightweight draft model to rapidly propose candidate tokens.
- **Verification Phase:** Uses the target model to verify multiple proposed tokens in parallel within a single step.
- **Impact:** Breaks sequential bottleneck while maintaining target distribution fidelity.

WHAT THESE METHODS IMPROVE

Latency & Throughput Speedups

Accelerates overall generation speed and increases GPU serving throughput significantly.

WHAT THEY DO NOT CHANGE

No Direct Quality / Accuracy Gain

They do not directly improve reasoning capability, output accuracy, or answer correctness.

TRAINING REASONING MODELS

Synthesis and Discussion

PROBABILISTIC LIMITS

Distribution vs. Correctness

Decoding chooses trajectories from a learned distribution. Probability alone does not establish correctness.

SYSTEM MECHANISM

Search & Verification

Sampling and search expose alternatives. Aggregation and verification determine how the system selects an answer.

COMPUTE ALLOCATION DISCUSSION

Discussion: When should a system spend more compute on sampling, verification, revision, or a larger base model?

Tradeoff Dimensions: **Sampling** • **Verification** • **Revision** • **Base Model Scale**

Advanced NLP - Session 1 - Advanced Reasoning & Mathematical Planning in LLMs – IUST

Reasoning Matters LLM to Reasoning Decoding